ig - SYNTAX ERROR

Striver

a2z

DSA SHEET

- "495"

By - SYNTAX ERROR

Step 17: Dynamic Programming

The PDF created by *Abhishek Rathor (Instagram:SYNTAX ERROR)*

PROBLEM: 1

Climbing Stairs

Level: Medium

Explanation:

This is a classic DP problem where you can either take 1 or 2 steps. The number of distinct ways to reach the top of the staircase with `n` steps is the same as the (n+1)th number in the Fibonacci sequence.

Time Complexity: O(n)

Space Complexity: O(1)

Java Code:

```
java
CopyEdit
public int climbStairs(int n) {
    if (n <= 2) return n;
    int first = 1, second = 2;
    for (int i = 3; i <= n; i++) {
        int temp = first + second;
        first = second;
        second = temp;
    }
    return second;
}
```

PROBLEM: 2

Frog Jump

Level: Medium

Explanation:

The frog is at the 0th stone and wants to reach the (n-1)th stone. The frog can jump either 1 or 2 steps ahead with a cost defined by the absolute difference in heights between the stones. Use DP to compute the minimum total cost.

Time Complexity: O(n)

Space Complexity: O(n)

Java Code:

```
java
CopyEdit
public int frogJump(int n, int[] heights) {
    int[] dp = new int[n];
    dp[0] = 0;
    for (int i = 1; i < n; i++) {
        int oneJump = dp[i - 1] + Math.abs(heights[i] - heights[i - 1]);
        int twoJump = (i > 1) ? dp[i - 2] + Math.abs(heights[i] - heights[i
- 2]) : Integer.MAX_VALUE;
        dp[i] = Math.min(oneJump, twoJump);
    }
    return dp[n - 1];
}
```

PROBLEM: 3

Frog Jump with K Distances

Level: Medium

Explanation:

The frog can jump from the current stone to any of the next `k` stones. The cost is the absolute height difference. The goal is to find the minimum cost to reach the last stone.

Time Complexity: O(n * k)

Space Complexity: O(n)

Java Code:

```
java
CopyEdit
public int frogJumpK(int n, int k, int[] heights) {
    int[] dp = new int[n];
    Arrays.fill(dp, Integer.MAX_VALUE);
    dp[0] = 0;
    for (int i = 1; i < n; i++) {
        for (int j = 1; j <= k; j++) {
            if (i - j >= 0) {
                int cost = dp[i - j] + Math.abs(heights[i] - heights[i -
j]);
                dp[i] = Math.min(dp[i], cost);
            }
        }
    }
    return dp[n - 1];
}
```

PROBLEM: 4

Maximum Sum of Non-Adjacent Elements

Level: Medium

Explanation:

You are given an array. Find the maximum sum of elements such that no two picked elements are adjacent. A variation of the House Robber problem.

Time Complexity: O(n)

Space Complexity: O(n)

Java Code:

```
java
CopyEdit
public int maxSumNonAdjacent(int[] nums) {
    int n = nums.length;
    if (n == 0) return 0;
    if (n == 1) return nums[0];
    int[] dp = new int[n];
```

```
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);
    for (int i = 2; i < n; i++) {
        dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);
    }
    return dp[n - 1];
}
```

PROBLEM: 5

House Robber

Level: Medium

Explanation:

Same as the previous problem with a real-world analogy: A robber can't rob two adjacent houses. Maximize the amount stolen.

Time Complexity: O(n)

Space Complexity: O(1) — Optimized

Java Code:

```
java
CopyEdit
public int rob(int[] nums) {
    int prev1 = 0, prev2 = 0;
    for (int num : nums) {
        int temp = Math.max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = temp;
    }
    return prev1;
}
```

Let me know if you want to continue with **PROBLEM: 6** and onwards.

4o

You said:

yes

ChatGPT said:

Great! Continuing from where we left off:

PROBLEM: 6

House Robber II

Level: Medium

Explanation:

Unlike House Robber I, the houses are arranged in a **circle**, which means the first and last houses are adjacent. You can't rob both. So, take the max of two scenarios:

- Rob from house 0 to n-2
- Rob from house 1 to n-1

Time Complexity: O(n)

Space Complexity: O(1)

Java Code:

```
java
CopyEdit
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];
    return Math.max(robRange(nums, 0, nums.length - 2), robRange(nums, 1,
nums.length - 1));
}

private int robRange(int[] nums, int start, int end) {
    int prev1 = 0, prev2 = 0;
    for (int i = start; i <= end; i++) {
        int temp = Math.max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = temp;
    }
    return prev1;
}
```

PROBLEM: 7

Ninja’s Training (DP on Grid)

Level: Medium

Explanation:

N days, 3 activities. Each day you can't do the same activity as the previous day. Maximize points earned. Use DP with last activity tracked.

Time Complexity: O(n * 4)

Space Complexity: O(n * 4)

Java Code:

```
java
CopyEdit
public int ninjaTraining(int n, int[][] points) {
    int[][] dp = new int[n][4];

    dp[0][0] = Math.max(points[0][1], points[0][2]);
    dp[0][1] = Math.max(points[0][0], points[0][2]);
    dp[0][2] = Math.max(points[0][0], points[0][1]);
    dp[0][3] = Math.max(points[0][0], Math.max(points[0][1], points[0][2]));

    for (int day = 1; day < n; day++) {
        for (int last = 0; last < 4; last++) {
            dp[day][last] = 0;
            for (int task = 0; task < 3; task++) {
                if (task != last) {
                    int point = points[day][task] + dp[day - 1][task];
                    dp[day][last] = Math.max(dp[day][last], point);
                }
            }
        }
    }
    return dp[n - 1][3];
}
```

PROBLEM: 8

Unique Paths

Level: Medium

Explanation:

Find the number of unique paths from top-left to bottom-right of an m x n grid. You can only move down or right.

Time Complexity: O(m * n)

Space Complexity: O(m * n)

Java Code:

```
java
CopyEdit
public int uniquePaths(int m, int n) {
    int[][] dp = new int[m][n];
    for (int i = 0; i < m; i++) dp[i][0] = 1;
    for (int j = 0; j < n; j++) dp[0][j] = 1;

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }
    return dp[m - 1][n - 1];
}
```

PROBLEM: 9

Unique Paths II (With Obstacles)

Level: Medium

Explanation:

Same as Unique Paths, but with some cells marked as obstacles (1). Return number of paths avoiding obstacles.

Time Complexity: O(m * n)

Space Complexity: O(m * n)

Java Code:

```
java
CopyEdit
public int uniquePathsWithObstacles(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int[][] dp = new int[m][n];

    if (grid[0][0] == 1) return 0;
    dp[0][0] = 1;

    for (int i = 1; i < m; i++)
        dp[i][0] = grid[i][0] == 0 ? dp[i - 1][0] : 0;

    for (int j = 1; j < n; j++)
        dp[0][j] = grid[0][j] == 0 ? dp[0][j - 1] : 0;

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            if (grid[i][j] == 0)
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }
    return dp[m - 1][n - 1];
}
```

PROBLEM: 10

Minimum Path Sum in Grid

Level: Medium

Explanation:

Given a grid filled with non-negative numbers, find a path from top-left to bottom-right which minimizes the sum of all numbers along its path.

Time Complexity: O(m * n)

Space Complexity: O(m * n)

Java Code:

```
java
CopyEdit
public int minPathSum(int[][] grid) {
```

```
    int m = grid.length, n = grid[0].length;
    int[][] dp = new int[m][n];
    dp[0][0] = grid[0][0];

    for (int i = 1; i < m; i++)
        dp[i][0] = dp[i - 1][0] + grid[i][0];

    for (int j = 1; j < n; j++)
        dp[0][j] = dp[0][j - 1] + grid[0][j];

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = Math.min(dp[i - 1][j], dp[i][j - 1]) + grid[i][j];
        }
    }
    return dp[m - 1][n - 1];
}
```

PROBLEM: 11

Minimum Path Sum in Triangle

Level: Medium

Explanation:

Given a triangle array, return the minimum path sum from top to bottom. Each step you may move to adjacent numbers on the row below.

Time Complexity: O(n²)

Space Complexity: O(n) — using optimized DP with 1D array

Java Code:

```
java
CopyEdit
public int minimumTotal(List<List<Integer>> triangle) {
    int n = triangle.size();
    int[] dp = new int[n];
    for (int i = 0; i < n; i++)
        dp[i] = triangle.get(n - 1).get(i);

    for (int i = n - 2; i >= 0; i--) {
        for (int j = 0; j <= i; j++) {
            dp[j] = triangle.get(i).get(j) + Math.min(dp[j], dp[j + 1]);
        }
    }
```

```
    return dp[0];
}
```

PROBLEM: 12

Minimum Falling Path Sum

Level: Medium

Explanation:

From any cell in the first row, move to the next row by going straight or diagonally left/right. Find the minimum sum path.

Time Complexity: O(n²)

Space Complexity: O(n²)

Java Code:

```
java
CopyEdit
public int minFallingPathSum(int[][] matrix) {
    int n = matrix.length;
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int left = j > 0 ? matrix[i - 1][j - 1] : Integer.MAX_VALUE;
            int mid = matrix[i - 1][j];
            int right = j < n - 1 ? matrix[i - 1][j + 1] :
Integer.MAX_VALUE;
            matrix[i][j] += Math.min(mid, Math.min(left, right));
        }
    }
    int min = Integer.MAX_VALUE;
    for (int j = 0; j < n; j++) min = Math.min(min, matrix[n - 1][j]);
    return min;
}
```

PROBLEM: 13

Cherry Pickup II

Level: Hard

Explanation:

Two robots start at top-left and top-right corners. They move down, collect cherries, and can move left/straight/right. Find the max cherries.

Time Complexity: O(m * n * n)

Space Complexity: O(m * n * n)

Java Code:

```
java
CopyEdit
public int cherryPickup(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int[][][] dp = new int[m][n][n];

    for (int j1 = 0; j1 < n; j1++) {
        for (int j2 = 0; j2 < n; j2++) {
            if (j1 == j2) dp[m - 1][j1][j2] = grid[m - 1][j1];
            else dp[m - 1][j1][j2] = grid[m - 1][j1] + grid[m - 1][j2];
        }
    }

    for (int i = m - 2; i >= 0; i--) {
        for (int j1 = 0; j1 < n; j1++) {
            for (int j2 = 0; j2 < n; j2++) {
                int max = 0;
                for (int d1 = -1; d1 <= 1; d1++) {
                    for (int d2 = -1; d2 <= 1; d2++) {
                        int nj1 = j1 + d1, nj2 = j2 + d2;
                        if (nj1 >= 0 && nj1 < n && nj2 >= 0 && nj2 < n) {
                            int val = dp[i + 1][nj1][nj2];
                            if (j1 == j2) val += grid[i][j1];
                            else val += grid[i][j1] + grid[i][j2];
                            max = Math.max(max, val);
                        }
                    }
                }
                dp[i][j1][j2] = max;
            }
        }
    }
    return dp[0][0][n - 1];
}
```

PROBLEM: 14

Ninja’s Training (DP 7)

Level: Medium

Explanation:

A ninja can perform 3 tasks daily but can't do the same task on consecutive days. Maximize the total merit points over `n` days.

Time Complexity: O(n * 4 * 3) = O(12n)

Space Complexity: O(4) – Optimized

Java Code:

```
java
CopyEdit
public int ninjaTraining(int n, int[][] points) {
    int[] prev = new int[4];

    prev[0] = Math.max(points[0][1], points[0][2]);
    prev[1] = Math.max(points[0][0], points[0][2]);
    prev[2] = Math.max(points[0][0], points[0][1]);
    prev[3] = Math.max(points[0][0], Math.max(points[0][1], points[0][2]));

    for (int day = 1; day < n; day++) {
        int[] temp = new int[4];
        for (int last = 0; last < 4; last++) {
            int max = 0;
            for (int task = 0; task < 3; task++) {
                if (task != last) {
                    int point = points[day][task] + prev[task];
                    max = Math.max(max, point);
                }
            }
            temp[last] = max;
        }
        prev = temp;
    }
    return prev[3];
}
```

PROBLEM: 15

Grid Unique Paths (DP 8)

Level: Medium

Explanation:

Find the number of unique paths from top-left to bottom-right in a grid, moving only right or down.

Time Complexity: O(m * n)

Space Complexity: O(m * n)

Java Code:

```
java
CopyEdit
public int uniquePaths(int m, int n) {
    int[][] dp = new int[m][n];
    for (int i = 0; i < m; i++) dp[i][0] = 1;
    for (int j = 0; j < n; j++) dp[0][j] = 1;

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }
    return dp[m - 1][n - 1];
}
```

PROBLEM: 16

Grid Unique Paths 2 (DP 9)

Level: Medium

Explanation:

Now the grid may contain obstacles. Return the number of unique paths avoiding obstacles.

Time Complexity: O(m * n)

Space Complexity: O(m * n)

Java Code:

```
java
CopyEdit
public int uniquePathsWithObstacles(int[][] obstacleGrid) {
    int m = obstacleGrid.length, n = obstacleGrid[0].length;
    int[][] dp = new int[m][n];

    if (obstacleGrid[0][0] == 1) return 0;
    dp[0][0] = 1;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (obstacleGrid[i][j] == 1) dp[i][j] = 0;
            else {
                if (i > 0) dp[i][j] += dp[i - 1][j];
                if (j > 0) dp[i][j] += dp[i][j - 1];
            }
        }
    }
    return dp[m - 1][n - 1];
}
```

PROBLEM: 17

Minimum Path Sum in Grid (DP 10)

Level: Medium

Explanation:

Return the minimum path sum from top-left to bottom-right. Only move right or down.

Time Complexity: O(m * n)

Space Complexity: O(m * n)

Java Code:

```
java
CopyEdit
public int minPathSum(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int[][] dp = new int[m][n];

    dp[0][0] = grid[0][0];
```

```
    for (int i = 1; i < m; i++) dp[i][0] = dp[i - 1][0] + grid[i][0];
    for (int j = 1; j < n; j++) dp[0][j] = dp[0][j - 1] + grid[0][j];

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = grid[i][j] + Math.min(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    return dp[m - 1][n - 1];
}
```

Sure! Continuing from:

PROBLEM: 18

Minimum Path Sum in Triangular Grid (DP 11)

Level: Medium

Explanation:

Given a triangle array, return the minimum path sum from top to bottom. Each step can move to adjacent numbers on the row below.

Time Complexity: O(n²)

Space Complexity: O(n)

Java Code:

```
public int minimumTotal(List<List<Integer>> triangle) {
    int n = triangle.size();
    int[] dp = new int[n];

    for (int i = 0; i < n; i++) {
        dp[i] = triangle.get(n - 1).get(i);
    }

    for (int i = n - 2; i >= 0; i--) {
        for (int j = 0; j <= i; j++) {
            dp[j] = triangle.get(i).get(j) + Math.min(dp[j], dp[j + 1]);
        }
    }
    return dp[0];
```

```
}
```

PROBLEM: 19

Minimum/Maximum Falling Path Sum (DP 12)

Level: Medium

Explanation:

Return the minimum path sum starting from any element in the first row to any element in the last row, where each move is directly below or diagonal.

Time Complexity: O(n²)

Space Complexity: O(n²)

Java Code:

```
public int minFallingPathSum(int[][] matrix) {
    int n = matrix.length;
    int[][] dp = new int[n][n];

    for (int j = 0; j < n; j++) dp[0][j] = matrix[0][j];

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int up = dp[i - 1][j];
            int leftDiag = j > 0 ? dp[i - 1][j - 1] : Integer.MAX_VALUE;
            int rightDiag = j < n - 1 ? dp[i - 1][j + 1] :
Integer.MAX_VALUE;
            dp[i][j] = matrix[i][j] + Math.min(up, Math.min(leftDiag,
rightDiag));
        }
    }

    int min = Integer.MAX_VALUE;
    for (int j = 0; j < n; j++) min = Math.min(min, dp[n - 1][j]);
    return min;
}
```

PROBLEM: 20

3D DP – Ninja and His Friends (DP 13)

Level: Medium

Explanation:

Two ninjas are collecting chocolates from a grid starting at (0,0) and (0,m-1) respectively and move to the bottom. They collect chocolates together if they land on the same cell.

Time Complexity: O(n * m * m)

Space Complexity: O(m * m)

Java Code:

```
public int ninjaChocolates(int[][] grid) {
    int n = grid.length, m = grid[0].length;
    int[][] front = new int[m][m];

    for (int j1 = 0; j1 < m; j1++) {
        for (int j2 = 0; j2 < m; j2++) {
            if (j1 == j2) front[j1][j2] = grid[n - 1][j1];
            else front[j1][j2] = grid[n - 1][j1] + grid[n - 1][j2];
        }
    }

    for (int i = n - 2; i >= 0; i--) {
        int[][] curr = new int[m][m];
        for (int j1 = 0; j1 < m; j1++) {
            for (int j2 = 0; j2 < m; j2++) {
                int max = Integer.MIN_VALUE;
                for (int dj1 = -1; dj1 <= 1; dj1++) {
                    for (int dj2 = -1; dj2 <= 1; dj2++) {
                        int nj1 = j1 + dj1, nj2 = j2 + dj2;
                        if (nj1 >= 0 && nj1 < m && nj2 >= 0 && nj2 < m) {
                            int val = (j1 == j2 ? grid[i][j1] : grid[i][j1]
+ grid[i][j2]) + front[nj1][nj2];
                            max = Math.max(max, val);
                        }
                    }
                }
                curr[j1][j2] = max;
            }
        }
        front = curr;
    }
    return front[0][m - 1];
}
```

PROBLEM: 21

Subset Sum Equal to Target (DP - 14)

Level: Medium

Explanation:

Given an array of integers and a target sum, return whether there's a subset whose sum equals the target.

Time Complexity: O(n * sum)

Space Complexity: O(sum)

Java Code:

```
java
CopyEdit
public boolean subsetSumToK(int n, int k, int[] arr) {
    boolean[] prev = new boolean[k + 1];
    prev[0] = true;

    if (arr[0] <= k) prev[arr[0]] = true;

    for (int i = 1; i < n; i++) {
        boolean[] curr = new boolean[k + 1];
        curr[0] = true;

        for (int target = 1; target <= k; target++) {
            boolean notTake = prev[target];
            boolean take = false;
            if (arr[i] <= target)
                take = prev[target - arr[i]];
            curr[target] = take || notTake;
        }

        prev = curr;
    }

    return prev[k];
}
```

PROBLEM: 22

Partition Equal Subset Sum (DP - 15)

Level: Medium

Explanation:

Check if the array can be partitioned into two subsets with equal sum.

Time Complexity: O(n * sum)

Space Complexity: O(sum)

Java Code:

```
java
CopyEdit
public boolean canPartition(int[] nums) {
    int total = 0;
    for (int num : nums) total += num;

    if (total % 2 != 0) return false;
    int target = total / 2;

    boolean[] dp = new boolean[target + 1];
    dp[0] = true;

    for (int num : nums) {
        for (int j = target; j >= num; j--) {
            dp[j] = dp[j] || dp[j - num];
        }
    }

    return dp[target];
}
```

PROBLEM: 23

Partition Set Into 2 Subsets With Min Absolute Sum Diff (DP - 16)

Level: Medium

Explanation:

Partition array into two subsets such that absolute difference between subset sums is minimized.

Time Complexity: O(n * sum)

Space Complexity: O(sum)

Java Code:

```
java
CopyEdit
public int minSubsetSumDifference(int[] nums, int n) {
    int totalSum = 0;
    for (int num : nums) totalSum += num;

    boolean[] prev = new boolean[totalSum + 1];
    prev[0] = true;

    for (int num : nums) {
        boolean[] curr = new boolean[totalSum + 1];
        for (int t = 0; t <= totalSum; t++) {
            curr[t] = prev[t] || (t >= num && prev[t - num]);
        }
        prev = curr;
    }

    int minDiff = Integer.MAX_VALUE;
    for (int s1 = 0; s1 <= totalSum / 2; s1++) {
        if (prev[s1]) {
            int s2 = totalSum - s1;
            minDiff = Math.min(minDiff, Math.abs(s2 - s1));
        }
    }

    return minDiff;
}
```

PROBLEM: 24

Count Subsets with Sum K (DP - 17)

Level: Medium

Explanation:

Given an array of `n` integers and an integer `k`, count the number of subsets whose sum is exactly `k`.

Time Complexity: O(n * k)

Space Complexity: O(k)

Java Code:

```
java
CopyEdit
public int findWays(int[] nums, int k) {
    int n = nums.length;
    int[] prev = new int[k + 1];
    prev[0] = 1;

    if (nums[0] <= k) prev[nums[0]] += 1;

    for (int i = 1; i < n; i++) {
        int[] curr = new int[k + 1];
        curr[0] = 1;

        for (int t = 0; t <= k; t++) {
            int notPick = prev[t];
            int pick = 0;
            if (nums[i] <= t) pick = prev[t - nums[i]];
            curr[t] = pick + notPick;
        }
        prev = curr;
    }
    return prev[k];
}
```

PROBLEM: 25

Count Partitions with Given Difference (DP - 18)

Level: Medium

Explanation:

Given an array and a difference `d`, count the number of ways to partition the array into two subsets such that the difference between subset sums is `d`.

Time Complexity: O(n * target)

Space Complexity: O(target)

Java Code:

```
java
```

```
CopyEdit
public int countPartitions(int n, int d, int[] arr) {
    int total = 0;
    for (int num : arr) total += num;

    if ((total - d) < 0 || (total - d) % 2 != 0) return 0;

    int target = (total - d) / 2;
    return findWays(arr, target);
}

public int findWays(int[] nums, int k) {
    int n = nums.length;
    int[] prev = new int[k + 1];
    prev[0] = 1;

    if (nums[0] <= k) prev[nums[0]] += 1;

    for (int i = 1; i < n; i++) {
        int[] curr = new int[k + 1];
        curr[0] = 1;

        for (int t = 0; t <= k; t++) {
            int notPick = prev[t];
            int pick = 0;
            if (nums[i] <= t) pick = prev[t - nums[i]];
            curr[t] = pick + notPick;
        }
        prev = curr;
    }
    return prev[k];
}
```

PROBLEM: 26

Assign Cookies

Level: Hard

Explanation:

Each child has a greed factor, and each cookie has a size. Assign cookies to children such that the number of content children is maximized.

Time Complexity: O(n log n + m log m)

Space Complexity: O(1)

Java Code:

```
java
CopyEdit
public int findContentChildren(int[] g, int[] s) {
    Arrays.sort(g);
    Arrays.sort(s);
    int child = 0, cookie = 0;

    while (child < g.length && cookie < s.length) {
        if (s[cookie] >= g[child]) child++;
        cookie++;
    }

    return child;
}
```

PROBLEM: 27

Minimum Coins (DP - 20)

Level: Hard

Explanation:

Given an array of coin denominations and a value `target`, find the minimum number of coins required to make the target. If not possible, return -1.

Time Complexity: O(n * target)

Space Complexity: O(target)

Java Code:

```
java
CopyEdit
public int coinChange(int[] coins, int target) {
    int n = coins.length;
    int[] dp = new int[target + 1];
    Arrays.fill(dp, (int)1e9);
    dp[0] = 0;

    for (int coin : coins) {
        for (int amt = coin; amt <= target; amt++) {
            dp[amt] = Math.min(dp[amt], 1 + dp[amt - coin]);
        }
    }
```

```
    return dp[target] == (int)1e9 ? -1 : dp[target];
}
```

PROBLEM: 28

Target Sum (DP - 21)

Level: Medium

Explanation:

Given an array of integers and a target value, return the number of ways to assign '+' or '-' signs to make the sum equal to the target.

Time Complexity: O(n * sum)

Space Complexity: O(sum)

Java Code:

```
java
CopyEdit
public int findTargetSumWays(int[] nums, int target) {
    int total = 0;
    for (int num : nums) total += num;

    if ((total + target) % 2 != 0 || total < Math.abs(target)) return 0;

    int sum = (total + target) / 2;
    return countSubsets(nums, sum);
}

private int countSubsets(int[] nums, int sum) {
    int[] dp = new int[sum + 1];
    dp[0] = 1;

    for (int num : nums) {
        for (int j = sum; j >= num; j--) {
            dp[j] += dp[j - num];
        }
    }

    return dp[sum];
}
```

Java Code:

```
java
CopyEdit
public int change(int target, int[] coins) {
    int[] dp = new int[target + 1];
    dp[0] = 1;

    for (int coin : coins) {
        for (int amt = coin; amt <= target; amt++) {
            dp[amt] += dp[amt - coin];
        }
    }

    return dp[target];
}
```

Time Complexity: O(n * W)

Space Complexity: O(W)

Java Code:

```
java
CopyEdit
public int unboundedKnapsack(int n, int W, int[] val, int[] wt) {
    int[] dp = new int[W + 1];

    for (int i = 0; i <= W; i++) {
        for (int j = 0; j < n; j++) {
            if (wt[j] <= i) {
                dp[i] = Math.max(dp[i], val[j] + dp[i - wt[j]]);
            }
        }
    }

    return dp[W];
}
```

Continuing with the next set of problems in the requested format:

PROBLEM: 34

Shortest Common Supersequence (DP - 27)

Level: Medium

Explanation:

Given two strings, return the length of the shortest string that has both strings as subsequences.

Time Complexity: O(m * n)

Space Complexity: O(m * n)

Java Code:

```
public int shortestCommonSupersequence(String str1, String str2) {
    int m = str1.length(), n = str2.length();
    int[][] dp = new int[m + 1][n + 1];

    // Find LCS length
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (str1.charAt(i - 1) == str2.charAt(j - 1))
                dp[i][j] = 1 + dp[i - 1][j - 1];
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    // Length of SCS = sum of lengths - length of LCS
    return m + n - dp[m][n];
}
```

PROBLEM: 35

Print Shortest Common Supersequence (DP - 28)

Level: Medium

Explanation:

Print the actual shortest common supersequence string of two given strings.

Time Complexity: O(m * n)

Space Complexity: O(m * n)

Java Code:

```
public String shortestCommonSupersequenceString(String str1, String str2) {
    int m = str1.length(), n = str2.length();
    int[][] dp = new int[m + 1][n + 1];

    // Compute LCS dp table
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (str1.charAt(i - 1) == str2.charAt(j - 1))
                dp[i][j] = 1 + dp[i - 1][j - 1];
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    // Build SCS using LCS table
    StringBuilder sb = new StringBuilder();
    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (str1.charAt(i - 1) == str2.charAt(j - 1)) {
            sb.append(str1.charAt(i - 1));
            i--; j--;
        } else if (dp[i - 1][j] > dp[i][j - 1]) {
            sb.append(str1.charAt(i - 1));
            i--;
        } else {
            sb.append(str2.charAt(j - 1));
            j--;
        }
    }

    while (i > 0) sb.append(str1.charAt(--i));
    while (j > 0) sb.append(str2.charAt(--j));

    return sb.reverse().toString();
}
```

PROBLEM: 36

Longest Palindromic Subsequence (DP - 29)

Level: Medium

Explanation:

Find the length of the longest subsequence that is also a palindrome.

Time Complexity: O(n * n)

Space Complexity: O(n * n)

Java Code:

```
public int longestPalindromeSubseq(String s) {
    String rev = new StringBuilder(s).reverse().toString();
    int n = s.length();
    int[][] dp = new int[n + 1][n + 1];

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (s.charAt(i - 1) == rev.charAt(j - 1))
                dp[i][j] = 1 + dp[i - 1][j - 1];
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    return dp[n][n];
}
```

PROBLEM: 37

Minimum Insertions to Make String Palindrome (DP - 30)

Level: Medium

Explanation:

Find the minimum number of insertions required to make the given string a palindrome.

Time Complexity: O(n * n)

Space Complexity: O(n * n)

Java Code:

```
java
CopyEdit
public int minInsertions(String s) {
    int n = s.length();
```

```
    String rev = new StringBuilder(s).reverse().toString();
    int[][] dp = new int[n + 1][n + 1];

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (s.charAt(i - 1) == rev.charAt(j - 1))
                dp[i][j] = 1 + dp[i - 1][j - 1];
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    return n - dp[n][n]; // Insertions = total length - LPS
}
```

PROBLEM: 38

Minimum Insertions/Deletions to Convert String A to B (DP - 31)

Level: Medium

Explanation:

Find the minimum number of insertions and deletions to convert one string into another.

Time Complexity: O(n * m)

Space Complexity: O(n * m)

Java Code:

```
java
CopyEdit
public int minOperations(String str1, String str2) {
    int m = str1.length(), n = str2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (str1.charAt(i - 1) == str2.charAt(j - 1))
                dp[i][j] = 1 + dp[i - 1][j - 1];
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    int lcs = dp[m][n];
    int deletions = m - lcs;
```

```
    int insertions = n - lcs;

    return deletions + insertions;
}
```

PROBLEM: 39

Shortest Common Supersequence (Print version) (DP - 32)

Level: Medium

Explanation:

Print the shortest string that has both `str1` and `str2` as subsequences.

Time Complexity: O(n * m)

Space Complexity: O(n * m)

Java Code:

```
java
CopyEdit
public String shortestCommonSupersequence(String str1, String str2) {
    int m = str1.length(), n = str2.length();
    int[][] dp = new int[m + 1][n + 1];

    // LCS DP table
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (str1.charAt(i - 1) == str2.charAt(j - 1))
                dp[i][j] = 1 + dp[i - 1][j - 1];
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    // Build answer
    StringBuilder sb = new StringBuilder();
    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (str1.charAt(i - 1) == str2.charAt(j - 1)) {
            sb.append(str1.charAt(i - 1));
            i--; j--;
        } else if (dp[i - 1][j] > dp[i][j - 1]) {
            sb.append(str1.charAt(i - 1));
            i--;
        } else {
```

```
            sb.append(str2.charAt(j - 1));
            j--;
        }
    }

    while (i > 0) sb.append(str1.charAt(--i));
    while (j > 0) sb.append(str2.charAt(--j));

    return sb.reverse().toString();
}
```

PROBLEM: 40

Distinct Subsequences (DP - 32)

Level: Hard

Explanation:

Count how many distinct subsequences of `s` equal the string `t`.

Time Complexity: O(n * m)

Space Complexity: O(n * m)

Java Code:

```
java
CopyEdit
public int numDistinct(String s, String t) {
    int n = s.length(), m = t.length();
    int[][] dp = new int[n + 1][m + 1];

    for (int i = 0; i <= n; i++) dp[i][0] = 1;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s.charAt(i - 1) == t.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
            } else {
                dp[i][j] = dp[i - 1][j];
            }
        }
    }

    return dp[n][m];
}
```

PROBLEM: 41

Edit Distance (DP - 33)

Level: Hard

Explanation:

Find the minimum number of operations (insert, delete, replace) required to convert string `word1` to `word2`.

Time Complexity: O(n * m)

Space Complexity: O(n * m)

Java Code:

```
java
CopyEdit
public int minDistance(String word1, String word2) {
    int n = word1.length(), m = word2.length();
    int[][] dp = new int[n + 1][m + 1];

    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + Math.min(
                    dp[i - 1][j - 1], // replace
                    Math.min(dp[i - 1][j], // delete
                             dp[i][j - 1]) // insert
                );
            }
        }
    }

    return dp[n][m];
}
```

PROBLEM: 42

Wildcard Matching (DP - 34)

Level: Hard

Explanation:

Check if the string matches a pattern where `'?'` matches any single character and `'*'` matches any sequence.

Time Complexity: O(n * m)

Space Complexity: O(n * m)

Java Code:

```
java
CopyEdit
public boolean isMatch(String s, String p) {
    int n = s.length(), m = p.length();
    boolean[][] dp = new boolean[n + 1][m + 1];

    dp[0][0] = true;

    for (int j = 1; j <= m; j++) {
        if (p.charAt(j - 1) == '*')
            dp[0][j] = dp[0][j - 1];
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (p.charAt(j - 1) == '?' || p.charAt(j - 1) == s.charAt(i -
1))
                dp[i][j] = dp[i - 1][j - 1];
            else if (p.charAt(j - 1) == '*')
                dp[i][j] = dp[i - 1][j] || dp[i][j - 1];
        }
    }

    return dp[n][m];
}
```

PROBLEM: 43

Regular Expression Matching (DP - 35)

Level: Hard

Explanation:

Support `.` which matches any single character and `*` which matches zero or more of the preceding element.

Time Complexity: O(n * m)

Space Complexity: O(n * m)

Java Code:

```
java
CopyEdit
public boolean isMatch(String s, String p) {
    int n = s.length(), m = p.length();
    boolean[][] dp = new boolean[n + 1][m + 1];
    dp[0][0] = true;

    for (int j = 1; j <= m; j++) {
        if (p.charAt(j - 1) == '*') {
            dp[0][j] = dp[0][j - 2];
        }
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (p.charAt(j - 1) == '.' || p.charAt(j - 1) == s.charAt(i -
1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else if (p.charAt(j - 1) == '*') {
                dp[i][j] = dp[i][j - 2] ||
                           ((p.charAt(j - 2) == '.' || p.charAt(j - 2) ==
s.charAt(i - 1)) && dp[i - 1][j]);
            }
        }
    }

    return dp[n][m];
}
```

PROBLEM: 44

Boolean Parenthesization (DP - 36)

Level: Hard

Explanation:

Count the number of ways to parenthesize the expression so that the result is true.

Time Complexity: O(n³)

Space Complexity: O(n² * 2)

Java Code:

```
java
CopyEdit
public int countWays(int N, String S) {
    int[][][] dp = new int[N][N][2];
    return count(S, 0, N - 1, 1, dp);
}

private int count(String s, int i, int j, int isTrue, int[][][] dp) {
    if (i > j) return 0;
    if (i == j) {
        if (isTrue == 1) return s.charAt(i) == 'T' ? 1 : 0;
        else return s.charAt(i) == 'F' ? 1 : 0;
    }

    if (dp[i][j][isTrue] != 0) return dp[i][j][isTrue];

    int ways = 0;
    for (int k = i + 1; k <= j - 1; k += 2) {
        int lt = count(s, i, k - 1, 1, dp);
        int lf = count(s, i, k - 1, 0, dp);
        int rt = count(s, k + 1, j, 1, dp);
        int rf = count(s, k + 1, j, 0, dp);

        char op = s.charAt(k);
        if (op == '&') {
            if (isTrue == 1) ways += lt * rt;
            else ways += lt * rf + lf * rt + lf * rf;
        } else if (op == '|') {
            if (isTrue == 1) ways += lt * rt + lt * rf + lf * rt;
            else ways += lf * rf;
        } else if (op == '^') {
            if (isTrue == 1) ways += lt * rf + lf * rt;
            else ways += lt * rt + lf * rf;
        }
    }

    return dp[i][j][isTrue] = ways % 1003;
}
```

PROBLEM: 45

Burst Balloons (DP - 37)

Level: Hard

Explanation:

Find the maximum coins by bursting balloons wisely.

Time Complexity: O(n³)

Space Complexity: O(n²)

Java Code:

```
java
CopyEdit
public int maxCoins(int[] nums) {
    int n = nums.length;
    int[] arr = new int[n + 2];
    for (int i = 0; i < n; i++) arr[i + 1] = nums[i];
    arr[0] = arr[n + 1] = 1;

    int[][] dp = new int[n + 2][n + 2];

    for (int len = 1; len <= n; len++) {
        for (int left = 1; left <= n - len + 1; left++) {
            int right = left + len - 1;
            for (int i = left; i <= right; i++) {
                dp[left][right] = Math.max(dp[left][right],
                    arr[left - 1] * arr[i] * arr[right + 1] +
                    dp[left][i - 1] + dp[i + 1][right]);
            }
        }
    }

    return dp[1][n];
}
```

PROBLEM: 46

Palindrome Partitioning II (DP - 38)

Level: Hard

Explanation:

Find the minimum number of cuts needed to partition a string such that every substring is a palindrome.

Time Complexity: O(n²)

Space Complexity: O(n²)

Java Code:

```
java
CopyEdit
public int minCut(String s) {
    int n = s.length();
    boolean[][] isPalin = new boolean[n][n];
    int[] dp = new int[n];

    for (int i = 0; i < n; i++) {
        int minCut = i;
        for (int j = 0; j <= i; j++) {
            if (s.charAt(i) == s.charAt(j) && (i - j < 2 || isPalin[j +
1][i - 1])) {
                isPalin[j][i] = true;
                minCut = j == 0 ? 0 : Math.min(minCut, dp[j - 1] + 1);
            }
        }
        dp[i] = minCut;
    }

    return dp[n - 1];
}
```

PROBLEM: 47

Maximum Rectangle in Binary Matrix (DP - 39)

Level: Hard

Explanation:

Convert each row to a histogram and use Largest Rectangle in Histogram logic.

Time Complexity: O(n * m)

Space Complexity: O(m)

Java Code:

```
java
CopyEdit
public int maximalRectangle(char[][] matrix) {
    if (matrix.length == 0) return 0;
    int maxArea = 0;
    int[] height = new int[matrix[0].length];

    for (char[] row : matrix) {
        for (int j = 0; j < row.length; j++) {
            height[j] = row[j] == '1' ? height[j] + 1 : 0;
        }
        maxArea = Math.max(maxArea, largestRectangleArea(height));
    }

    return maxArea;
}

private int largestRectangleArea(int[] heights) {
    Stack<Integer> stack = new Stack<>();
    int max = 0;
    int[] newHeights = Arrays.copyOf(heights, heights.length + 1);

    for (int i = 0; i < newHeights.length; i++) {
        while (!stack.isEmpty() && newHeights[i] < newHeights[stack.peek()])
{
            int h = newHeights[stack.pop()];
            int w = stack.isEmpty() ? i : i - 1 - stack.peek();
            max = Math.max(max, h * w);
        }
        stack.push(i);
    }

    return max;
}
```

PROBLEM: 48

Largest Square of 1's (DP - 40)

Level: Hard

Explanation:

Use DP to build up the size of the square ending at each point.

Time Complexity: O(n * m)

Space Complexity: O(n * m)

Java Code:

```
java
CopyEdit
public int maximalSquare(char[][] matrix) {
    int max = 0;
    int n = matrix.length, m = matrix[0].length;
    int[][] dp = new int[n][m];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (matrix[i][j] == '1') {
                if (i == 0 || j == 0)
                    dp[i][j] = 1;
                else
                    dp[i][j] = Math.min(Math.min(dp[i - 1][j], dp[i][j -
1]), dp[i - 1][j - 1]) + 1;
                max = Math.max(max, dp[i][j]);
            }
        }
    }

    return max * max;
}
```

PROBLEM: 49

Count Square Submatrices with All Ones (DP - 41)

Level: Medium

Explanation:

Use dynamic programming similar to maximal square, but accumulate count of all square submatrices formed at each cell.

Time Complexity: O(n * m)

Space Complexity: O(n * m)

Java Code:

```
java
```

```
CopyEdit
public int countSquares(int[][] matrix) {
    int n = matrix.length, m = matrix[0].length;
    int[][] dp = new int[n][m];
    int total = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (matrix[i][j] == 1) {
                if (i == 0 || j == 0)
                    dp[i][j] = 1;
                else
                    dp[i][j] = Math.min(Math.min(dp[i - 1][j], dp[i][j -
1]), dp[i - 1][j - 1]) + 1;
                total += dp[i][j];
            }
        }
    }

    return total;
}
```

PROBLEM: 50

Boolean Parenthesization Problem (DP - 42)

Level: Hard

Explanation:

Use memoization with multiple states: start, end, and boolean value to store results of subproblems.

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$

Java Code:

```
java
CopyEdit
Map<String, Integer> memo = new HashMap<>();

public int countWays(int n, String s) {
    return count(s, 0, n - 1, true);
}

private int count(String s, int i, int j, boolean isTrue) {
```

```
    if (i > j) return 0;
    if (i == j) {
        if (isTrue) return s.charAt(i) == 'T' ? 1 : 0;
        else return s.charAt(i) == 'F' ? 1 : 0;
    }

    String key = i + "_" + j + "_" + isTrue;
    if (memo.containsKey(key)) return memo.get(key);

    int ways = 0;
    for (int k = i + 1; k <= j - 1; k += 2) {
        char op = s.charAt(k);

        int lt = count(s, i, k - 1, true);
        int lf = count(s, i, k - 1, false);
        int rt = count(s, k + 1, j, true);
        int rf = count(s, k + 1, j, false);

        if (op == '&') {
            ways += isTrue ? lt * rt : lt * rf + lf * rt + lf * rf;
        } else if (op == '|') {
            ways += isTrue ? lt * rt + lt * rf + lf * rt : lf * rf;
        } else if (op == '^') {
            ways += isTrue ? lt * rf + lf * rt : lt * rt + lf * rf;
        }
    }

    memo.put(key, ways);
    return ways;
}
```

PROBLEM: 55

Maximum Rectangle Area with all 1's (DP - 55)

Level: Hard

Explanation:

Transform the problem into a series of largest rectangle in histogram problems by updating row-wise heights and applying the histogram logic for each row.

Time Complexity: O(n * m)

Space Complexity: O(m)

Java Code:

```
java
```

```
CopyEdit
public int maximalRectangle(char[][] matrix) {
    if (matrix.length == 0) return 0;
    int maxArea = 0;
    int cols = matrix[0].length;
    int[] heights = new int[cols];

    for (char[] row : matrix) {
        for (int i = 0; i < cols; i++) {
            heights[i] = row[i] == '1' ? heights[i] + 1 : 0;
        }
        maxArea = Math.max(maxArea, largestRectangleArea(heights));
    }

    return maxArea;
}

private int largestRectangleArea(int[] heights) {
    Stack<Integer> stack = new Stack<>();
    int max = 0;
    int i = 0;
    while (i <= heights.length) {
        int h = (i == heights.length) ? 0 : heights[i];
        if (stack.isEmpty() || h >= heights[stack.peek()]) {
            stack.push(i++);
        } else {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - 1 - stack.peek();
            max = Math.max(max, height * width);
        }
    }
    return max;
}
```

PROBLEM: 56

Count Square Submatrices with All Ones (DP - 56)

Level: Medium

Explanation:

DP approach where for each cell, the value represents the size of the largest square ending at that cell. The total count is the sum of all these values.

Time Complexity: O(n * m)

Space Complexity: O(n * m)

Java Code:

```
java
CopyEdit
public int countSquares(int[][] matrix) {
    int n = matrix.length, m = matrix[0].length;
    int[][] dp = new int[n][m];
    int total = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (matrix[i][j] == 1) {
                if (i == 0 || j == 0)
                    dp[i][j] = 1;
                else
                    dp[i][j] = Math.min(Math.min(dp[i - 1][j], dp[i][j -
1]), dp[i - 1][j - 1]) + 1;
                total += dp[i][j];
            }
        }
    }

    return total;
}
```

Step 17: Tries

PROBLEM: 59

Implement Trie - 2 (Prefix Tree)

Level: Hard

Explanation:

This variation supports:

- Insert a word
- Count the number of times a word appears
- Count the number of words starting with a given prefix
- Erase one occurrence of a word

Time Complexity:

- Insert: O(L)
- Count: O(L)
- Erase: O(L)
  where L = length of the word

Java Code:

```
java
CopyEdit
class TrieNode {
    TrieNode[] links = new TrieNode[26];
    int end = 0, prefix = 0;

    boolean containsKey(char ch) {
        return links[ch - 'a'] != null;
    }

    TrieNode get(char ch) {
        return links[ch - 'a'];
    }

    void put(char ch, TrieNode node) {
        links[ch - 'a'] = node;
    }
}

class Trie {
    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {
        TrieNode node = root;
        for (char ch : word.toCharArray()) {
            if (!node.containsKey(ch)) {
                node.put(ch, new TrieNode());
            }
            node = node.get(ch);
            node.prefix++;
        }
        node.end++;
    }

    public int countWordsEqualTo(String word) {
        TrieNode node = root;
        for (char ch : word.toCharArray()) {
            if (!node.containsKey(ch)) return 0;
            node = node.get(ch);
        }
        return node.end;
    }

    public int countWordsStartingWith(String prefix) {
        TrieNode node = root;
        for (char ch : prefix.toCharArray()) {
            if (!node.containsKey(ch)) return 0;
            node = node.get(ch);
        }
        return node.prefix;
    }
```

```
    public void erase(String word) {
        TrieNode node = root;
        for (char ch : word.toCharArray()) {
            if (!node.containsKey(ch)) return;
            node = node.get(ch);
            node.prefix--;
        }
        node.end--;
    }
}
```

PROBLEM: 60

Longest String with All Prefixes

Level: Medium

Explanation:

From a list of strings, return the longest string such that all its prefixes are present in the list.

Time Complexity: O(n * L)

Space Complexity: O(n * L)

Where L is the average length of strings.

Java Code:

```
java
CopyEdit
class TrieNode {
    TrieNode[] links = new TrieNode[26];
    boolean end = false;

    boolean containsKey(char ch) {
        return links[ch - 'a'] != null;
    }

    TrieNode get(char ch) {
        return links[ch - 'a'];
    }

    void put(char ch, TrieNode node) {
        links[ch - 'a'] = node;
    }
}
```

```
class Trie {
    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {
        TrieNode node = root;
        for (char ch : word.toCharArray()) {
            if (!node.containsKey(ch)) {
                node.put(ch, new TrieNode());
            }
            node = node.get(ch);
        }
        node.end = true;
    }

    public boolean allPrefixExists(String word) {
        TrieNode node = root;
        for (char ch : word.toCharArray()) {
            if (!node.containsKey(ch)) return false;
            node = node.get(ch);
            if (!node.end) return false;
        }
        return true;
    }
}

public String longestWord(String[] words) {
    Trie trie = new Trie();
    for (String word : words) {
        trie.insert(word);
    }

    String ans = "";
    for (String word : words) {
        if (trie.allPrefixExists(word)) {
            if (word.length() > ans.length() ||
                (word.length() == ans.length() && word.compareTo(ans) < 0))
{
                ans = word;
            }
        }
    }
    return ans;
}
```

PROBLEM: 61

Number of Distinct Substrings in a String

Level: Hard

Explanation:

You are given a string. Return the total number of distinct substrings in the string, including the empty string.

Time Complexity: O(n²)

Space Complexity: O(n²)

Using Trie-based brute force approach.

Java Code:

```
java
CopyEdit
class TrieNode {
    TrieNode[] children = new TrieNode[26];
}

public class Solution {
    public int countDistinctSubstrings(String s) {
        TrieNode root = new TrieNode();
        int count = 0;

        for (int i = 0; i < s.length(); i++) {
            TrieNode node = root;
            for (int j = i; j < s.length(); j++) {
                char ch = s.charAt(j);
                if (node.children[ch - 'a'] == null) {
                    node.children[ch - 'a'] = new TrieNode();
                    count++; // new substring formed
                }
                node = node.children[ch - 'a'];
            }
        }

        return count + 1; // including the empty string
    }
}
```

PROBLEM: 62

Bit PreRequisites for TRIE Problems

Level: Hard

Explanation:

Before solving XOR problems using Trie, you should understand:

- Binary representation of integers
- Bit manipulation: shifting, masking
- How to represent numbers in binary format using Trie

Practice Code Snippet:

```
java
CopyEdit
public class BitManipulationDemo {
    public static void main(String[] args) {
        int num = 10;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            System.out.print(bit);
        }
    }
}
```

PROBLEM: 63

Maximum XOR of Two Numbers in an Array

Level: Medium

Explanation:

Given an integer array `nums`, return the maximum result of `nums[i] XOR nums[j]` where `0 <= i, j < n`.

This is solved using a Trie based on the binary representation of numbers.

Time Complexity: O(32 * N)

Space Complexity: O(32 * N)

Java Code:

```
java
CopyEdit
class TrieNode {
    TrieNode[] children = new TrieNode[2];
```

```
}

public class Solution {
    TrieNode root = new TrieNode();

    public void insert(int num) {
        TrieNode node = root;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (node.children[bit] == null) {
                node.children[bit] = new TrieNode();
            }
            node = node.children[bit];
        }
    }

    public int findMaxXOR(int num) {
        TrieNode node = root;
        int maxXor = 0;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (node.children[1 - bit] != null) {
                maxXor |= (1 << i);
                node = node.children[1 - bit];
            } else {
                node = node.children[bit];
            }
        }
        return maxXor;
    }

    public int findMaximumXOR(int[] nums) {
        for (int num : nums) {
            insert(num);
        }
        int max = 0;
        for (int num : nums) {
            max = Math.max(max, findMaxXOR(num));
        }
        return max;
    }
}
```

PROBLEM: 64

Maximum XOR With an Element From Array

Level: Medium

Explanation:

You are given an array `nums`, and a list of queries. For each query, return the maximum XOR of the given number with any element in the array such that the element is less than or equal to the given constraint.

Use offline queries + Trie + sorting.

Time Complexity: O((N + Q) * 32 + QlogQ + NlogN)

Space Complexity: O(N * 32)

Java Code:

```
java
CopyEdit
class TrieNode {
    TrieNode[] children = new TrieNode[2];
}

public class Solution {
    TrieNode root = new TrieNode();

    public void insert(int num) {
        TrieNode node = root;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (node.children[bit] == null)
                node.children[bit] = new TrieNode();
            node = node.children[bit];
        }
    }

    public int maxXOR(int x) {
        TrieNode node = root;
        int maxXor = 0;
        for (int i = 31; i >= 0; i--) {
            int bit = (x >> i) & 1;
            if (node.children[1 - bit] != null) {
                maxXor |= (1 << i);
                node = node.children[1 - bit];
            } else {
                node = node.children[bit];
            }
        }
        return maxXor;
    }

    public int[] maximizeXor(int[] nums, int[][] queries) {
        Arrays.sort(nums);
        int q = queries.length;
        int[][] newQueries = new int[q][3];

        for (int i = 0; i < q; i++) {
            newQueries[i][0] = queries[i][0];
            newQueries[i][1] = queries[i][1];
            newQueries[i][2] = i;
        }

        Arrays.sort(newQueries, Comparator.comparingInt(a -> a[1]));
```

```
        int[] result = new int[q];
        int index = 0;
        for (int[] query : newQueries) {
            int x = query[0], m = query[1], i = query[2];
            while (index < nums.length && nums[index] <= m) {
                insert(nums[index]);
                index++;
            }

            if (index == 0) {
                result[i] = -1;
            } else {
                result[i] = maxXOR(x);
            }
        }

        return result;
    }
}
```

Step 18: Strings Hard

Problem 65: Minimum number of bracket reversals needed to make an expression balanced
Level: Medium

```
java
CopyEdit
public int minReversals(String expr) {
    int len = expr.length();
    if (len % 2 != 0) return -1;

    Stack<Character> stack = new Stack<>();
    for (char ch : expr.toCharArray()) {
        if (ch == '{') {
            stack.push(ch);
        } else {
            if (!stack.isEmpty() && stack.peek() == '{') {
                stack.pop();
            } else {
                stack.push(ch);
            }
        }
    }

    int open = 0, close = 0;
    while (!stack.isEmpty()) {
        if (stack.pop() == '{') open++;
        else close++;
    }

    return (open + 1) / 2 + (close + 1) / 2;
}
```

Problem 66: Count and Say
Level: Medium

```
java
CopyEdit
public String countAndSay(int n) {
    if (n == 1) return "1";
    String prev = countAndSay(n - 1);
    StringBuilder sb = new StringBuilder();
    int count = 1;
    for (int i = 1; i < prev.length(); i++) {
        if (prev.charAt(i) == prev.charAt(i - 1)) {
            count++;
        } else {
            sb.append(count).append(prev.charAt(i - 1));
            count = 1;
        }
    }
    sb.append(count).append(prev.charAt(prev.length() - 1));
    return sb.toString();
}
```

Problem 67: Rabin Karp
Level: Hard

```
java
CopyEdit
public List<Integer> rabinKarp(String text, String pattern) {
    List<Integer> result = new ArrayList<>();
    if (pattern.length() > text.length()) return result;

    int base = 256;
    int mod = 101;
    int n = text.length(), m = pattern.length();
    int h = 1;

    for (int i = 0; i < m - 1; i++) {
        h = (h * base) % mod;
    }

    int pHash = 0, tHash = 0;
    for (int i = 0; i < m; i++) {
        pHash = (base * pHash + pattern.charAt(i)) % mod;
        tHash = (base * tHash + text.charAt(i)) % mod;
    }

    for (int i = 0; i <= n - m; i++) {
        if (pHash == tHash && text.substring(i, i + m).equals(pattern)) {
            result.add(i);
        }
        if (i < n - m) {
            tHash = (base * (tHash - text.charAt(i) * h) + text.charAt(i +
m)) % mod;
            if (tHash < 0) tHash += mod;
        }
    }

    return result;
}
```

Problem 68: Z-Function
Level: Easy

```
java
CopyEdit
public int[] zFunction(String s) {
    int n = s.length();
    int[] z = new int[n];
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) {
            z[i] = Math.min(r - i + 1, z[i - l]);
        }
        while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) {
            z[i]++;
        }
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}
```

Problem 69: KMP Algorithm / LPS (Longest Prefix Suffix) Array
Level: Hard

```
java
CopyEdit
public int[] computeLPSArray(String pattern) {
    int n = pattern.length();
    int[] lps = new int[n];
    int len = 0, i = 1;
    while (i < n) {
        if (pattern.charAt(i) == pattern.charAt(len)) {
            lps[i++] = ++len;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i++] = 0;
            }
        }
    }
    return lps;
}
```

Problem 70: Shortest Palindrome
Level: Hard

```
java
CopyEdit
public String shortestPalindrome(String s) {
    String rev = new StringBuilder(s).reverse().toString();
    String combined = s + "#" + rev;
    int[] lps = computeLPSArray(combined);
    int len = lps[lps.length - 1];
```

```
    return new StringBuilder(s.substring(len)).reverse().toString() + s;
}
```

Problem 71: Longest Happy Prefix
Level: Hard

```
java
CopyEdit
public String longestPrefix(String s) {
    int[] lps = computeLPSArray(s);
    int len = lps[s.length() - 1];
    return s.substring(0, len);
}
```

Problem 72: Count Palindromic Subsequences in Given String
Level: Hard

```
java
CopyEdit
public int countPalindromicSubsequences(String s) {
    int n = s.length();
    int[][] dp = new int[n][n];
    int mod = 1000000007;

    for (int i = 0; i < n; i++) dp[i][i] = 1;

    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;
            if (s.charAt(i) == s.charAt(j)) {
                dp[i][j] = (dp[i + 1][j] + dp[i][j - 1] + 1) % mod;
            } else {
                dp[i][j] = (dp[i + 1][j] + dp[i][j - 1] - dp[i + 1][j - 1]
+ mod) % mod;
            }
        }
    }

    return dp[0][n - 1];
}
```

Problem 73: Implement Trie (Prefix Tree)
Level: Hard

```
java
CopyEdit
class TrieNode {
    TrieNode[] links = new TrieNode[26];
    boolean isEnd = false;

    boolean containsKey(char ch) {
        return links[ch - 'a'] != null;
    }

    TrieNode get(char ch) {
        return links[ch - 'a'];
```

```
    }

    void put(char ch, TrieNode node) {
        links[ch - 'a'] = node;
    }

    void setEnd() {
        isEnd = true;
    }

    boolean isEnd() {
        return isEnd;
    }
}

class Trie {
    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {
        TrieNode node = root;
        for (char ch : word.toCharArray()) {
            if (!node.containsKey(ch)) {
                node.put(ch, new TrieNode());
            }
            node = node.get(ch);
        }
        node.setEnd();
    }

    public boolean search(String word) {
        TrieNode node = searchPrefix(word);
        return node != null && node.isEnd();
    }

    public boolean startsWith(String prefix) {
        return searchPrefix(prefix) != null;
    }

    private TrieNode searchPrefix(String word) {
        TrieNode node = root;
        for (char ch : word.toCharArray()) {
            if (node.containsKey(ch)) {
                node = node.get(ch);
            } else {
                return null;
            }
        }
        return node;
    }
}
```

Problem 74: Longest String with All Prefixes
Level: Medium

java

```
CopyEdit
public String longestWord(String[] words) {
    Set<String> set = new HashSet<>(Arrays.asList(words));
    Arrays.sort(words);
    String result = "";
    for (String word : words) {
        boolean allPrefixExist = true;
        for (int i = 1; i < word.length(); i++) {
            if (!set.contains(word.substring(0, i))) {
                allPrefixExist = false;
                break;
            }
        }
        if (allPrefixExist && word.length() > result.length()) {
            result = word;
        }
    }
    return result;
}
```

Problem 75: Number of Distinct Substrings in a String
Level: Hard

```
java
CopyEdit
class TrieNode {
    TrieNode[] children = new TrieNode[26];
}

public int countDistinctSubstrings(String s) {
    TrieNode root = new TrieNode();
    int count = 0;
    for (int i = 0; i < s.length(); i++) {
        TrieNode node = root;
        for (int j = i; j < s.length(); j++) {
            char ch = s.charAt(j);
            if (node.children[ch - 'a'] == null) {
                node.children[ch - 'a'] = new TrieNode();
                count++;
            }
            node = node.children[ch - 'a'];
        }
    }
    return count + 1; // +1 for empty substring
}
```

Congratulations on completing the PDF created by** SYNTAX ERROR**, also known as Abhishek Rathor! Your dedication and perseverance are truly commendable. For more insights and updates, feel free to connect on Instagram at** SYNTAX ERROR**.